

Sprint 12 Full Structured Notes: Performance Foundations, Browser Rendering, INP, and Sustainability

0. Where You Are Before Sprint 12

Before Sprint 12, you should already understand these React ideas:

- React is a JavaScript library for building user interfaces.
- Vite helps you create and run React projects.
- JSX lets you write UI-like syntax inside JavaScript.
- Components are reusable pieces of UI.
- Props pass data from parent components to child components.
- `children` lets one component wrap other JSX.
- Conditional rendering lets React show different UI based on data.
- `map()` renders lists.
- Stable keys help React track list items.
- Events let React respond to clicks, typing, and form submissions.
- Controlled inputs store form values in React state.
- `useState` lets React remember changing data.
- State updates should be immutable.
- `useReducer` can organize related state logic.
- `useRef` can access DOM elements or store values that should not cause re-renders.
- `useEffect` runs side effects after React renders.
- Cleanup functions stop old timers, listeners, and subscriptions.
- Custom hooks let you reuse hook logic.
- Context API lets you share app-level values like user, theme, or language.
- Data fetching lets React communicate with APIs or backend services.
- Loading, data, and error states make async UI clear.
- `useMemo`, `useCallback`, and `React.memo` help reduce unnecessary work in some cases.
- React Router connects URLs to React components.
- Dynamic routes, nested routes, 404 routes, protected routes, and lazy-loaded routes help make React apps feel like multi-page applications.
- Sprint 10 introduced shared state, prop drilling, single source of truth, Context, Redux, Zustand, and React DevTools.
- Sprint 11 introduced CSS frameworks and preprocessors.

So far, your main question has been:

How do I make a React app work correctly?

Sprint 12 starts a new question:

How do I make a React app load, render, and respond in a way that feels fast?

Important distinction:

Sprint 8 already introduced React-level optimization tools such as `useMemo`, `useCallback`, and `React.memo`. Sprint 12 is broader. It is not mainly about memoization. It is about how the browser turns your app into pixels, why pages feel slow, why user interactions can feel delayed, and how performance affects real users.

1. Sprint 12 Goal

The goal of Sprint 12 is to understand performance foundations.

By the end of Sprint 12, you should be able to:

- Explain real performance.
- Explain perceived performance.
- Explain why an app can technically load but still feel slow.
- Explain the browser rendering process.
- Explain DOM, CSSOM, render tree, layout, paint, and compositing.
- Explain the critical rendering path.
- Understand why source order matters.
- Understand why render-blocking resources matter.
- Use `defer` for non-critical scripts.
- Explain layout shift.
- Reserve image space with `width` and `height`.
- Use lazy loading for non-critical images.
- Explain latency.
- Explain what a CDN does.
- Explain compression, caching, and image optimization at a beginner level.
- Explain INP, or Interaction to Next Paint.
- Understand how heavy JavaScript can make a page feel frozen.
- Give users fast feedback during slow actions.
- Understand the link between performance and sustainability.
- Build a small Performance Notes page.
- Use Chrome DevTools or Lighthouse at a beginner level to observe performance.

Simple version:

Sprint 12 teaches you that a React app is not finished just because it works. It also needs to load, render, and respond well.

2. Big Mental Model

A React app goes through many steps before the user can actually use it.

Simple flow:

```
User opens app
↓
Browser downloads HTML, CSS, and JavaScript
↓
Browser builds internal page structures
↓
React runs
↓
Browser calculates layout
↓
Browser paints pixels
↓
User sees the page
↓
User clicks or types
↓
JavaScript and React respond
↓
Browser paints the updated UI
```

Performance is about making this whole flow efficient.

Beginner idea:

Good performance means the user sees useful content quickly and gets a fast response when they interact.

3. What Is Web Performance?

Web performance means how well a website or app loads, displays, and responds.

A website with good performance usually:

- Shows important content quickly.
- Responds quickly to clicks and typing.
- Avoids unexpected layout jumps.
- Does not freeze during interaction.
- Does not download unnecessary large files.

- Works reasonably well on slower devices and networks.
- Gives feedback while waiting.

A website with poor performance may:

- Show a blank screen for too long.
- Load a huge JavaScript bundle before showing content.
- Move content around while images load.
- Freeze when the user clicks a button.
- Use large uncompressed images.
- Make too many network requests.
- Waste battery, CPU, and bandwidth.

Beginner rule:

Performance is not only about speed numbers. It is also about user experience.

4. Real Performance

4.1 What Is Real Performance?

Real performance means measurable speed.

It uses numbers.

Examples:

```
The page loaded in 1.8 seconds.  
The server responded in 200ms.  
The JavaScript bundle is 300KB.  
The button response took 120ms.  
The image file is 80KB.
```

Simple definition:

Real performance is performance you can measure with tools.

4.2 Examples of Real Performance Metrics

Common measurable things include:

- Load time.
- Server response time.
- JavaScript bundle size.
- CSS file size.

- Image file size.
- Number of network requests.
- Rendering time.
- Interaction delay.
- Time until content appears.
- Time until the page becomes usable.

4.3 Tools for Measuring Real Performance

Beginner tools include:

```
Chrome DevTools
Lighthouse
Network tab
Performance tab
PageSpeed Insights
```

You do not need to master all tools now.

For Sprint 12, understand this:

Tools help you see what is slow instead of guessing.

5. Perceived Performance

5.1 What Is Perceived Performance?

Perceived performance means how fast the app feels to the user.

Simple definition:

Perceived performance is the user's feeling of speed.

A page can take a few seconds to finish loading but still feel acceptable if it gives useful feedback early.

A page can also load technically fast but feel bad if the user sees nothing or the app freezes.

5.2 Bad Perceived Performance

Bad experience:

```
User clicks Save
↓
```

```
Nothing happens
↓
User waits
↓
Maybe the result appears later
```

The user may think:

```
Did the click work?
Is the app broken?
Should I click again?
```

5.3 Good Perceived Performance

Better experience:

```
User clicks Save
↓
Button changes to Saving...
↓
Button becomes disabled
↓
Result appears
↓
Button changes back
```

The user understands what is happening.

5.4 Beginner Code Example

Good feedback pattern:

```
<button disabled={isSaving}>
  {isSaving ? "Saving..." : "Save"}
</button>
```

This is better than:

```
<button>Save</button>
```

when saving takes time.

5.5 Beginner Rule

Do not leave the user guessing. Show feedback during slow work.

6. Real Performance vs Perceived Performance

Topic	Meaning	Example
Real performance	Measurable speed	Page loaded in 2 seconds
Perceived performance	How fast it feels	User sees a skeleton screen quickly

Both matter.

A good app should:

- Actually be efficient.
- Feel responsive and trustworthy.

Simple example:

Real performance problem:
The JavaScript file is too large.

Perceived performance problem:
The user sees no loading message while waiting.

Beginner rule:

Measure real performance, but also design for how the user experiences waiting.

7. Browser Rendering: How Code Becomes Pixels

When you open a website, the browser does not magically display the page.

It follows a process.

Simple rendering flow:

HTML → DOM
CSS → CSSOM
DOM + CSSOM → Render Tree

Render Tree → Layout
Layout → Paint
Paint → Composite
Composite → Screen

This process matters because slow HTML, CSS, JavaScript, images, or layout work can delay what the user sees.

8. Step 1: HTML Becomes the DOM

8.1 What Is HTML?

HTML describes the structure of a page.

Example:

```
<h1>Hello</h1>  
<p>Welcome</p>
```

This tells the browser:

- There is a heading.
- There is a paragraph.

8.2 What Is the DOM?

DOM means Document Object Model.

Simple definition:

The DOM is the browser's structured representation of your HTML.

The browser turns HTML into a tree-like structure.

Example:

```
document  
├── html  
│   ├── body  
│   │   ├── h1  
│   │   └── p
```


React eventually updates the real DOM when state or props change.

8.3 Beginner Rule

HTML is the text you write. DOM is the structure the browser builds from that HTML.

9. Step 2: CSS Becomes the CSSOM

9.1 What Is CSS?

CSS describes how the page should look.

Example:

```
h1 {  
  color: blue;  
}
```

This tells the browser:

Make `h1` text blue.

9.2 What Is the CSSOM?

CSSOM means CSS Object Model.

Simple definition:

The CSSOM is the browser's structured representation of your CSS rules.

The browser needs the CSSOM to know how each visible element should be styled.

9.3 Why CSS Can Affect Performance

CSS can affect performance because the browser often needs CSS before it can correctly display the page.

Large or blocking CSS can delay rendering.

Beginner rule:

The browser needs both structure and style before it can display the page correctly.

10. Step 3: DOM + CSSOM Become the Render Tree

The browser combines the DOM and CSSOM.

Simple idea:

```
HTML structure
+
CSS styles
=
Visible styled elements
```

This creates the render tree.

Simple definition:

The render tree is the browser's plan for what visible elements need to be drawn.

10.1 Not Everything Is in the Render Tree

If an element is hidden with `display: none`, it usually does not appear in the render tree.

Example:

```
.hidden {
  display: none;
}
```

That element exists in the DOM, but it does not need to be drawn on the screen.

10.2 Beginner Rule

The DOM stores page structure. The render tree stores what needs to be visibly rendered.

11. Step 4: Layout

11.1 What Is Layout?

Layout means the browser calculates size and position.

The browser answers questions like:

How wide is this card?
How tall is this image?
Where does this button sit?
How much space does this paragraph need?

Simple definition:

Layout is the browser calculating where everything goes and how much space it takes.

Layout is also called reflow.

11.2 Why Layout Can Be Expensive

Layout can become expensive when the page keeps changing sizes or positions.

Examples:

- Images load without reserved space.
- JavaScript keeps changing element sizes.
- Fonts load and change text dimensions.
- Large lists appear all at once.
- Complex layouts keep recalculating.

11.3 Beginner Rule

Avoid unnecessary layout changes after the page starts loading.

12. Step 5: Paint

12.1 What Is Paint?

Paint means the browser fills pixels.

The browser draws things like:

text
colors
borders
backgrounds
shadows
images

Simple definition:

Paint is the step where the browser draws visual details.

12.2 Why Paint Can Be Expensive

Painting can become expensive when the page has many visual effects.

Examples:

- Large shadows.
- Many gradients.
- Huge images.
- Complex backgrounds.
- Many elements changing often.

12.3 Beginner Rule

More visual complexity can mean more work for the browser.

13. Step 6: Compositing

13.1 What Is Compositing?

Compositing means the browser combines layers into the final screen.

Simple definition:

Compositing combines painted layers into the final image the user sees.

Some things may create layers, such as:

- Fixed headers.
- Transformed elements.
- Certain animations.
- Overlays.
- Modals.

13.2 Beginner Rule

The browser may paint parts separately and then combine them into the final screen.

14. Browser Rendering Summary

Full flow:

```
HTML parsed
↓
DOM created
↓
CSS parsed
↓
CSSOM created
↓
DOM + CSSOM combined
↓
Render tree created
↓
Layout calculates positions and sizes
↓
Paint draws pixels
↓
Composite combines layers
↓
User sees the page
```

Short memory rule:

```
DOM = structure
CSSOM = styles
Render tree = visible styled elements
Layout = size and position
Paint = pixels
Composite = final screen
```

15. Critical Rendering Path

15.1 What Is the Critical Rendering Path?

The critical rendering path is the minimum work the browser must do before the user sees useful content.

Simple definition:

The critical rendering path is the browser's path from receiving files to showing the first useful pixels.

15.2 Beginner Mental Model

Important code first.
Unimportant code later.

The first screen should not wait for unnecessary scripts, images, or styles.

15.3 Bad Example

```
<script src="large-analytics-file.js"></script>  
<h1>Welcome</h1>  
<p>Main content</p>
```

Problem:

The browser may handle the large script before showing important content.

15.4 Better Example

```
<h1>Welcome</h1>  
<p>Main content appears early.</p>  
<script src="large-analytics-file.js" defer></script>
```

Why this is better:

- Important content appears earlier in the document.
- The non-critical script is deferred.
- The browser can parse HTML before running the script.

15.5 Beginner Rule

Make the browser's first important work as small and direct as possible.

16. Source Order

16.1 What Is Source Order?

Source order means the order of content in your HTML or JSX.

Example:

```
<main>
  <h1>Dashboard</h1>
  <p>Important summary appears first.</p>
  <LargeChart />
</main>
```

Better than:

```
<main>
  <LargeChart />
  <h1>Dashboard</h1>
  <p>Important summary appears later.</p>
</main>
```

If the chart is heavy, the important message may be delayed.

16.2 Beginner Rule

Put the most important content early.

17. Progressive Enhancement

17.1 What Is Progressive Enhancement?

Progressive enhancement means starting with a basic working experience, then adding extra features when possible.

Simple definition:

Progressive enhancement means the core content should still make sense even before all advanced JavaScript finishes.

Example idea:

- Basic text appears first.
- Then interactive React features load.
- Then advanced charts or animations appear.

17.2 Why It Matters

Progressive enhancement helps users on:

- Slower networks.

- Older phones.
- Low-power devices.
- Browsers with delayed JavaScript loading.

Beginner rule:

Do not make basic understanding depend on every advanced script loading first.

18. Render-Blocking Resources

18.1 What Is Render Blocking?

A render-blocking resource delays the browser from showing the page.

Common examples:

- Large CSS files.
- Synchronous JavaScript files.
- Scripts placed before important content.

18.2 Why JavaScript Can Block Rendering

JavaScript can change the DOM and CSS.

Because of that, the browser may pause HTML parsing to run JavaScript.

Example:

```
<script src="big-file.js"></script>
```

If this script is large and not deferred, it may delay the first visible content.

18.3 Beginner Rule

Do not block the first screen with scripts that are not needed immediately.

19. The `defer` Attribute

19.1 What Does `defer` Do?

Example:


```
<script src="analytics.js" defer></script>
```

Simple definition:

`defer` downloads the script without blocking HTML parsing and runs it after the document has been parsed.

19.2 When to Use `defer`

Use `defer` for scripts that are not needed immediately to show the first screen.

Examples:

- Analytics scripts.
- Non-critical widgets.
- Extra behavior that can wait.

19.3 Beginner Rule

If a script is not needed to show the first useful content, consider deferring it.

20. Layout Shift

20.1 What Is Layout Shift?

Layout shift happens when content moves unexpectedly after the page starts loading.

Simple definition:

Layout shift is unexpected movement of page content.

20.2 Bad Example

```

```

Problem:

The browser may not know the image size before it loads.

When the image finally loads, it can push other content down.

20.3 Better Example

```

```

Why this is better:

```
width and height reserve space  
alt helps accessibility  
loading="lazy" delays off-screen loading  
.webp may reduce file size
```

20.4 Beginner Rule

Reserve space for images so the page does not jump when they load.

21. Image Optimization

21.1 Why Images Matter

Images are often some of the largest files on a website.

Large images can cause:

- Slow loading.
- More bandwidth usage.
- Slower rendering.
- Poor experience on mobile.
- Higher energy use.

21.2 Beginner Image Optimization Habits

Use these habits:

- Use the correct image size.
- Avoid uploading huge images when a smaller one is enough.
- Use modern formats like WebP when appropriate.
- Add `width` and `height`.

- Use meaningful `alt` text.
- Use `loading="lazy"` for non-critical off-screen images.

21.3 Example

```

```

21.4 Beginner Rule

Do not make users download more image data than they need.

22. Lazy Loading

22.1 What Is Lazy Loading?

Lazy loading means loading something only when it is needed.

Simple definition:

Lazy loading delays non-critical content until later.

Examples:

- Images below the first screen.
- Large route pages.
- Heavy components.
- Optional charts.

22.2 Image Lazy Loading

```

```

This tells the browser:

Do not load this image immediately if it is not needed yet.

22.3 React Lazy Loading Reminder

In Sprint 9, you learned route/component lazy loading:

```
import { lazy, Suspense } from "react";

const Dashboard = lazy(() => import("./pages/Dashboard"));
```

Then:

```
<Suspense fallback={<p>Loading dashboard...</p>}>
  <Dashboard />
</Suspense>
```

That is React component lazy loading.

Sprint 12 connects the idea to performance:

Load less upfront. Load more when needed.

23. Latency

23.1 What Is Latency?

Latency means travel time.

Simple definition:

Latency is the delay while data travels between the user and another system.

Example:

```
Browser sends request
↓
Request travels to server
↓
Server handles request
↓
Response travels back
↓
Browser receives response
```

Even if your React code is good, a far-away or slow server can make the app feel slow.

23.2 Causes of Latency

Latency can come from:

- Long physical distance to the server.
- Slow network connection.
- Slow server response.
- Too many requests.
- Large files.
- Missing caching.

23.3 Beginner Rule

Performance is not only frontend code. Network distance and server response also matter.

24. CDN

24.1 What Is a CDN?

CDN means Content Delivery Network.

Simple definition:

A CDN serves files from locations closer to the user.

Without a CDN:

User in Sri Lanka → server in the US → files travel far

With a CDN:

User in Sri Lanka → nearby CDN location → files travel less

24.2 What CDNs Are Commonly Used For

CDNs often serve:

- Images.
- CSS files.
- JavaScript files.
- Fonts.

- Static assets.

24.3 Beginner Rule

A CDN can reduce waiting time by serving static files from a closer location.

25. Compression

25.1 What Is Compression?

Compression means making files smaller for transfer.

Simple definition:

Compression reduces file size so downloads are faster.

Examples:

- Compressing JavaScript.
- Compressing CSS.
- Compressing HTML.
- Compressing images.

25.2 Why Compression Matters

Smaller files usually mean:

- Faster downloads.
- Less bandwidth usage.
- Better mobile experience.
- Less energy waste.

25.3 Beginner Rule

Smaller files usually travel faster.

26. Caching

26.1 What Is Caching?

Caching means saving a copy of a resource so it does not need to be downloaded again every time.

Simple definition:

Caching lets the browser reuse files it already has.

Example:

```
First visit:  
Download app.css  
  
Second visit:  
Reuse cached app.css
```

26.2 Why Caching Matters

Caching can make repeat visits faster.

It helps with:

- Static assets.
- Images.
- Fonts.
- CSS.
- JavaScript bundles.

26.3 Beginner Rule

If a file does not change often, the browser should not need to download it again every time.

27. Time to Usable

27.1 What Is Time to Usable?

Time to usable means how long it takes before the user can meaningfully use the page.

Simple definition:

Time to usable is when the page is not only visible, but usable.

A page may show content but still be hard to use if JavaScript is busy.

27.2 Improving Time to Usable

You can improve it by:

- Showing important content early.
- Deferring non-critical JavaScript.

- Lazy loading large components.
- Avoiding heavy work during startup.
- Keeping first-page bundles smaller.
- Showing loading states.

27.3 Beginner Rule

The user should not wait for everything before they can use something important.

28. INP: Interaction to Next Paint

28.1 What Is INP?

INP means Interaction to Next Paint.

Simple definition:

INP measures how long it takes after a user interaction before the page visually responds.

Example interaction:

- User clicks a button.
- User types into an input.
- User opens a menu.
- User selects an option.

28.2 INP Flow

```
User clicks button
↓
Browser receives interaction
↓
Event handler runs
↓
JavaScript does work
↓
React may update state
↓
Browser renders updated UI
↓
User sees visual response
```

INP measures the delay in this process.

28.3 Beginner Interpretation

Good INP:

```
User clicks
↓
UI responds quickly
```

Poor INP:

```
User clicks
↓
Page feels stuck
↓
UI responds late
```

28.4 General INP Thresholds

Beginner memory:

```
Good: under about 200ms
Needs improvement: around 200ms to 500ms
Poor: above about 500ms
```

Do not obsess over exact numbers yet.

For now, understand this:

Lower interaction delay usually feels better.

29. Why Heavy JavaScript Hurts Interaction

29.1 The Main Thread

Browsers run much page work on the main thread.

The main thread may handle:

- JavaScript.
- Event handlers.
- Style calculation.
- Layout.

- Paint preparation.

If JavaScript runs for too long, the browser may not respond quickly to user input.

29.2 Bad Example

```
export default function App() {  
  function handleSearch() {  
    let total = 0;  
  
    for (let i = 0; i < 2_000_000_000; i++) {  
      total += i;  
    }  
  
    console.log(total);  
  }  
  
  return (  
    <button type="button" onClick={handleSearch}>  
      Search  
    </button>  
  );  
}
```

Problem:

The click starts heavy JavaScript.

While that JavaScript runs, the browser cannot quickly update the UI.

The user may feel:

I clicked, but nothing happened.

29.3 Beginner Rule

Long JavaScript work can block the browser from responding.

30. Fast Feedback During Interaction

30.1 Why Feedback Matters

If something takes time, show the user that the app received their action.

Examples of feedback:

- Button text changes to `Saving...`.
- Submit button becomes disabled.
- Loading message appears.
- Skeleton UI appears.
- Progress message appears.
- Result area updates.

30.2 Beginner Example

```
import { useState } from "react";

export default function App() {
  const [status, setStatus] = useState("idle");

  function handleSearch() {
    setStatus("Searching...");

    setTimeout(() => {
      setStatus("Search complete");
    }, 500);
  }

  return (
    <main>
      <button type="button" onClick={handleSearch}>
        Search
      </button>

      <p>{status}</p>
    </main>
  );
}
```

This is not a full advanced optimization.

It teaches the beginner idea:

Tell the user what is happening.

30.3 Beginner Rule

If the app cannot finish immediately, it should respond immediately.

31. Skeleton Screens and Loading UI

31.1 What Is a Loading UI?

Loading UI tells the user that work is happening.

Examples:

```
Loading posts...  
Saving...  
Preparing dashboard...  
Fetching products...
```

31.2 What Is a Skeleton Screen?

A skeleton screen is a placeholder layout that looks like the shape of the content that is coming.

Simple definition:

A skeleton screen shows the structure of the page before the real data arrives.

Example idea:

```
[ gray title placeholder ]  
[ gray paragraph placeholder ]  
[ gray card placeholder ]
```

31.3 Why Skeletons Help Perceived Performance

Skeletons help because the user sees progress and page structure quickly.

The app feels more alive than a blank screen.

31.4 Beginner Rule

A blank screen feels slower than a clear loading state.

32. Avoiding Layout Jumps

32.1 Common Causes of Layout Jumps

Layout jumps can happen because of:

- Images without width and height.
- Ads or embeds loading late.
- Fonts changing after load.
- Content inserted above existing content.
- Error messages appearing without reserved space.
- Large lazy-loaded sections appearing suddenly.

32.2 Better Pattern

Reserve space when possible.

Example:

```
<section className="image-card">
  
</section>
```

32.3 Beginner Rule

Do not surprise the user by moving content under their eyes.

33. Performance and Accessibility

Performance and accessibility are connected.

A slow or jumpy page can hurt users who rely on:

- Keyboard navigation.
- Screen readers.
- Magnification.
- Older devices.
- Slower networks.

Examples:

- If content shifts, a keyboard user may lose their place.
- If buttons are delayed, users may activate something twice.
- If loading states are missing, screen reader users may not know what changed.

Beginner rule:

A stable, responsive app is usually more accessible.

34. Sustainability and Performance

34.1 Why Performance Affects Sustainability

Performance is not only about user comfort.

Heavy websites can waste:

CPU work
battery
network bandwidth
server resources
energy

34.2 A Lighter App Is Usually Better

A lighter app is usually:

- Faster.
- Cheaper to serve.
- Better for low-end devices.
- Better for battery life.
- Better for users on limited data.
- Better for sustainability.

34.3 Beginner Sustainability Habits

Good habits:

- Use smaller images.
- Avoid unnecessary scripts.
- Remove unused CSS and JavaScript.
- Cache files when appropriate.
- Use efficient algorithms.
- Avoid excessive animations.

- Do not fetch the same data repeatedly for no reason.
- Lazy load non-critical content.

34.4 Beginner Rule

Less unnecessary work usually means better performance and less wasted energy.

35. Common Beginner Performance Problems

Problem 1: Huge Images

Bad:

```

```

Better:

```

```

Problem 2: No Loading State

Bad:

```
<button type="button" onClick={handleSave}>Save</button>
```

Better:

```
<button type="button" disabled={isSaving} onClick={handleSave}>  
  {isSaving ? "Saving..." : "Save"}  
</button>
```

Problem 3: Heavy Work Inside Click Handler

Bad:

```
function handleClick() {  
  for (let i = 0; i < 2_000_000_000; i++) {  
    // heavy work  
  }  
}
```

Better beginner approach:

- Give feedback first.
- Avoid unnecessary heavy work.
- Move expensive work away from immediate interaction when possible.
- Split large work when appropriate.

Problem 4: Script Blocks Important Content

Bad:

```
<script src="large-non-critical.js"></script>  
<h1>Welcome</h1>
```

Better:

```
<h1>Welcome</h1>  
<script src="large-non-critical.js" defer></script>
```

Problem 5: Layout Jumps

Bad:

```

```

Better:

```

```

36. Beginner Chrome DevTools Performance Checks

You do not need to become a performance expert yet.

Start with basic checks.

36.1 Network Tab

Use the Network tab to ask:

- Which files are loading?
- Which files are large?
- How many requests happen?
- Are images too large?
- Is JavaScript too large?
- Is anything loading slowly?

36.2 Performance Tab

Use the Performance tab to ask:

- What happens when the page loads?
- What happens when I click a button?
- Is JavaScript taking a long time?
- Is rendering delayed?
- Are there long tasks?

36.3 Lighthouse

Use Lighthouse to get a beginner report about:

- Performance.
- Accessibility.
- Best practices.
- SEO.

Do not treat the score as the only goal.

Use it to find problems.

Beginner rule:

Tools are for finding causes, not just chasing scores.

37. Sprint 12 Mini Project: Performance Notes Page

37.1 Project Goal

Build a small page called **Performance Notes**.

The goal is to practice:

- Important content early.
- Real vs perceived performance explanations.
- Lazy image loading.
- Reserved image dimensions.
- Fast feedback on interaction.
- A basic heavy-work demo.

37.2 Files

Create:

```
src/App.jsx  
src/App.css
```

38. Mini Project Code: `src/App.jsx`

```
import { useState } from "react";  
import "./App.css";  
  
export default function App() {  
  const [feedback, setFeedback] = useState("No interaction yet.");  
  const [heavyResult, setHeavyResult] = useState(null);  
  
  function handleFastFeedback() {  
    setFeedback("Button responded immediately.");  
  }  
  
  function handleHeavyWork() {  
    setFeedback("Running heavy work...");  
  
    let total = 0;  
  
    for (let i = 0; i < 50_000_000; i++) {  
      total += i;  
    }  
  
    setHeavyResult(total);  
    setFeedback("Heavy work finished.");  
  }  
  
  return (  
    <main className="page">
```

```

<section className="hero">
  <h1>Performance Notes</h1>
  <p>
    A React app should not only work correctly. It should also load,
    render, and respond quickly.
  </p>
</section>

<section className="card">
  <h2>Real Performance</h2>
  <p>
    Real performance is measured with numbers, such as load time,
    response time, and interaction delay.
  </p>
</section>

<section className="card">
  <h2>Perceived Performance</h2>
  <p>
    Perceived performance is how fast the app feels to the user. Loading
    messages and instant feedback improve the feeling.
  </p>

  <button type="button" onClick={handleFastFeedback}>
    Give Fast Feedback
  </button>

  <p className="status">{feedback}</p>
</section>

<section className="card">
  <h2>Lazy Image</h2>
  <p>
    This image reserves space with width and height, and it uses lazy
    loading.
  </p>

  
</section>

<section className="card">

```

```

    <h2>Interaction Delay Demo</h2>
    <p>
      Click this button and observe whether the page feels delayed. Then
      record it in Chrome DevTools Performance tab.
    </p>

    <button type="button" onClick={handleHeavyWork}>
      Run Heavy Work
    </button>

    {heavyResult !== null && <p>Result: {heavyResult}</p>}
  </section>
</main>
);
}

```

39. Mini Project Code: `src/App.css`

```

* {
  box-sizing: border-box;
}

body {
  margin: 0;
  font-family: system-ui, sans-serif;
  background: #f5f5f5;
  color: #222;
}

.page {
  max-width: 900px;
  margin: 0 auto;
  padding: 24px;
}

.hero {
  margin-bottom: 24px;
}

.card {
  background: white;
  border: 1px solid #ddd;
  border-radius: 12px;
  padding: 16px;
}

```

```
    margin-bottom: 16px;
  }

  button {
    border: none;
    border-radius: 8px;
    padding: 10px 14px;
    cursor: pointer;
  }

  .status {
    font-weight: 600;
  }

  .preview {
    display: block;
    max-width: 100%;
    height: auto;
    border-radius: 8px;
  }
}
```

40. Mini Project Explanation

40.1 Important Content Early

```
<section className="hero">
  <h1>Performance Notes</h1>
  <p>
    A React app should not only work correctly. It should also load,
    render, and respond quickly.
  </p>
</section>
```

This puts the main message at the top.

Beginner idea:

Show important content early.

40.2 Fast Feedback Button

```
function handleFastFeedback() {  
  setFeedback("Button responded immediately.");  
}
```

This demonstrates perceived performance.

The user clicks and sees an immediate response.

40.3 Lazy Image

```

```

This shows four good habits:

- `src` gives the image source.
- `width` and `height` reserve layout space.
- `alt` helps accessibility.
- `loading="lazy"` delays loading if the image is not immediately needed.

40.4 Heavy Work Demo

```
for (let i = 0; i < 50_000_000; i++) {  
  total += i;  
}
```

This intentionally creates work so you can observe delay.

Beginner warning:

This is a demo. Do not add heavy loops to real UI interactions unless there is a clear reason.

41. Debug Task

Fix this code:

```

<script src="large-non-critical.js"></script>
<button onclick="doVeryHeavyWork()">Search</button>
```

41.1 Better HTML Version

```


<script src="large-non-critical.js" defer></script>

<button type="button" id="search-button">
  Search
</button>
```

41.2 Problems Fixed

Problem	Fix
Image had no reserved size	Added <code>width</code> and <code>height</code>
Image may be larger than needed	Used <code>.webp</code> idea
Image loaded immediately	Added <code>loading="lazy"</code>
Script could block rendering	Added <code>defer</code>
Inline handler is not React style	In React, use <code>onClick={handleSearch}</code>

41.3 React Version

```
function SearchButton() {
  function handleSearch() {
    console.log("Search started");
  }
}
```

```

return (
  <button type="button" onClick={handleSearch}>
    Search
  </button>
);
}

```

42. Common Sprint 12 Vocabulary

Term	Simple Meaning
Real performance	Speed measured with numbers
Perceived performance	How fast the app feels
DOM	Browser structure created from HTML
CSSOM	Browser structure created from CSS
Render tree	Visible styled elements the browser plans to draw
Layout	Browser calculates size and position
Reflow	Another name often used for layout recalculation
Paint	Browser draws pixels
Composite	Browser combines layers into final screen
Critical rendering path	Work needed before useful content appears
Render-blocking	Something that delays the page from rendering
<code>defer</code>	Lets a script wait until HTML is parsed
Layout shift	Unexpected movement of content
Lazy loading	Loading something only when needed
Latency	Network travel delay
CDN	Network that serves files from locations closer to users
Compression	Making files smaller for transfer
Caching	Reusing files instead of downloading again
INP	Delay from interaction to next visual update
Sustainability	Reducing wasted CPU, network, battery, and energy use

43. Revision Checkpoint Questions

Answer these without looking at the notes:

1. What is real performance?
 2. What is perceived performance?
 3. Why can an app feel slow even if it technically loads?
 4. What is the DOM?
 5. What is the CSSOM?
 6. What is the render tree?
 7. What happens during layout?
 8. What happens during paint?
 9. What happens during compositing?
 10. What is the critical rendering path?
 11. Why does source order matter?
 12. What is a render-blocking resource?
 13. What does `defer` do?
 14. What is layout shift?
 15. Why should images have `width` and `height`?
 16. What does `loading="lazy"` do?
 17. What is latency?
 18. What does a CDN do?
 19. Why does compression improve performance?
 20. What is caching?
 21. What does INP measure?
 22. Why can heavy JavaScript make a button feel slow?
 23. What is one way to improve perceived performance?
 24. How can performance affect accessibility?
 25. How can better performance reduce wasted energy?
-

44. Mandatory Practice Coding Block

44.1 Warm-Up Drills

Do these first:

1. Write one sentence that defines real performance.
2. Write one sentence that defines perceived performance.
3. Write the browser rendering flow from memory.
4. Write an image tag with `width`, `height`, `alt`, and `loading="lazy"`.
5. Write a button that changes text to `Saving...` when `isSaving` is true.

44.2 Guided Coding Task

Build the **Performance Notes** page using:

- `src/App.jsx`
- `src/App.css`

Your page must include:

- A heading.
- A short explanation of real performance.
- A short explanation of perceived performance.
- A lazy-loaded image with reserved dimensions.
- A button that gives instant feedback.
- A heavy-work demo button.

44.3 Debug This Code

Fix the performance problems:

```
<script src="big-widget.js"></script>

<button onclick="runSlowCode()">Click</button>
```

Expected improvements:

- Important content should not be blocked by non-critical script.
- Image should reserve space.
- Image should be optimized.
- Off-screen image should be lazy-loaded.
- React should use `onClick`, not inline HTML `onclick`.

44.4 Build Something Small

Build a **Product Landing Page** with:

- A hero section.
- Three product cards.
- Lazy-loaded product images.
- A loading state for a fake subscribe button.
- A short note explaining how you avoided layout shift.

44.5 Challenge Task

Use Chrome DevTools Performance tab:

1. Record your app loading.

2. Click the heavy-work button.
3. Observe whether the UI feels delayed.
4. Write down what happened.
5. Reduce the loop size and test again.

Goal:

Learn to observe performance instead of guessing.

45. Expected Output / Self-Check

By the end of Sprint 12, your app should:

- Show important content immediately.
 - Explain real performance in simple words.
 - Explain perceived performance in simple words.
 - Use at least one image with `width` and `height`.
 - Use `loading="lazy"` where appropriate.
 - Give instant feedback after a button click.
 - Demonstrate how heavy JavaScript can delay interaction.
 - Use semantic HTML such as `main`, `section`, `h1`, `h2`, and `button`.
 - Avoid unnecessary layout shift.
 - Be understandable without a blank screen.
-

46. Common Beginner Mistakes

Mistake 1: Thinking Performance Is Only About React

Wrong idea:

My React code works, so performance is fine.

Better idea:

The browser, network, JavaScript, CSS, images, and user feedback all affect performance.

Mistake 2: Ignoring Loading States

Wrong idea:

The request finishes eventually, so no loading message is needed.

Better idea:

Users need feedback while waiting.

Mistake 3: Using Huge Images

Wrong idea:

Any image file is fine if it displays correctly.

Better idea:

Image size, format, dimensions, and lazy loading matter.

Mistake 4: Forgetting Image Dimensions

Wrong idea:

The browser will figure out the image size later.

Better idea:

Reserve image space to avoid layout shift.

Mistake 5: Blocking the First Screen

Wrong idea:

Load every script immediately.

Better idea:

Load critical things first and delay non-critical things.

Mistake 6: Ignoring Interaction Delay

Wrong idea:

The click handler eventually finishes, so it is fine.

Better idea:

If the UI does not respond quickly, the app feels broken.

47. Sprint 12 Retrospective Prompts

After finishing Sprint 12, answer these:

1. Which concept was easiest: real performance, perceived performance, rendering, or INP?
 2. Which part of browser rendering is still unclear?
 3. Did the heavy-work demo make the UI feel delayed?
 4. Did adding instant feedback improve the user experience?
 5. Did you remember to add image dimensions?
 6. What would you optimize first in a real React app?
 7. What would you measure before changing code?
 8. What would you refactor if this app became larger?
-

48. Sprint 12 Shortest Mental Model

Sprint 12 teaches that performance is not only a React issue. The browser must turn HTML, CSS, and JavaScript into pixels. If important content is delayed, images shift layout, JavaScript blocks interaction, or users get no feedback, the app feels slow. A good React developer understands both real speed and perceived speed.

49. Final Beginner Rules

Remember these rules:

1. A working app is not automatically a fast app.
2. Real performance is measured with numbers.
3. Perceived performance is how fast the app feels.
4. The browser builds DOM, CSSOM, render tree, layout, paint, and composite steps.
5. Put important content early.
6. Defer non-critical scripts.
7. Reserve image space with `width` and `height`.
8. Lazy load non-critical images.
9. Avoid heavy JavaScript during user interactions.
10. Give feedback immediately when something takes time.
11. Use tools to measure instead of guessing.
12. Better performance also helps accessibility, battery life, bandwidth, and sustainability.